

作业 HW6 实验报告

姓名：庄子硕 学号：2350998 日期：2025 年 2 月 28 日

- # 实验报告格式要求按照模板（使用 Markdown 等也请保证报告内包含模板中的要素）
- # 对字体大小、缩进、颜色等不做强制要求（但尽量代码部分和文字内容有一定区分，可参考 vscode 配色）
- # 实验报告要求在文字简洁的同时将内容表示清楚
- # 报告内不要大段贴代码，尽量控制在 20 页以内

1. 涉及数据结构和相关背景

排序

2. 实验内容

2.1 求逆序对数

2.1.1 问题描述

对于一个长度为 N 的整数序列 A ，满足 $i < j$ 且 $A_i > A_j$ 的数对 (i,j) 称为整数序列 A 的一个逆序。请求出整数序列 A 的所有逆序对个数

2.1.2 基本要求

输入

输入包含多组测试数据，每组测试数据有两行
第一行为整数 N ($1 \leq N \leq 20000$)，当输入 0 时结束
第二行为 N 个整数，表示长为 N 的整数序列

输出

每组数据对应一行，输出逆序对的个数

2.1.3 数据结构设计

```
int a[20010]; // 原数据
int b[20010]; // 排序后数据
int cnt=0; // 逆序对数
```

2.1.4 功能说明（函数、类）

```
// 合并两个有序序列
void combine(int l,int r)
{
    int mid=(l+r)/2;
```

```

    int i=1;
    int j=mid+1;

    for(int k=1;k<=r;k++)
    {
        if(j>r|| (i<=mid&& a[i]<=a[j])) b[k]=a[i++];
        else {
            b[k]=a[j++];
            cnt+=mid-i+1;
        }
    }
    for(int i=1;i<=r;i++)
        a[i]=b[i];
}

// 归并排序
void merge_sort(int l,int r)
{
    int mid=(l+r)/2;

    if(l==r)return;
    else{
        merge_sort(l,mid);
        merge_sort(mid+1,r);
    }

    combine(l,r);
}

```

2.1.5 调试分析（遇到的问题 and 解决方法）

问题：小规模数据正常运行，大规模不行

解决方法：因为排序中的数列不能被逐步生成的有序序列覆盖，从而需要将有序序列在合并后覆盖；否则在递归回到上层时会认为已经排序好，若不覆盖则没有排序。

2.1.6 总结和体会

求逆序对数，其实就是排序的过程；排序就是在消灭逆序对；

2.2 最大数

2.2.1 问题描述

给定一组非负整数 `nums`，重新排列每个数的顺序（每个数不可拆分）使之组成一个最大的整数。

注意：输出结果可能非常大，所以你需要返回一个字符串而不是整数。

2.2.2 基本要求

输入包含两行 第一行包含一个整数 `n`，表示组数 `nums` 的长度

第二行包含 `n` 个整数 `nums[i]`

输出包含一行，为重新排列后得到的数字

2.2.3 数据结构设计

```
// 字符串数组
String ss[110];
```

2.2.4 功能说明（函数、类）

```
// 贪心排序函数
bool comp(string s1,string s2)
{
    return s1+s2 >s2+s1;
}
```

2.2.5 调试分析（遇到的问题 and 解决方法）

问题：本地可以，oj 不行

解决方案：没有使用 `template`，使用就可以了

2.2.6 总结和体会

要证明贪心算法的正确性；要证明 `comp` 的传递性

2.3 排序

2.3.1 问题描述

排序算法分为简单排序（时间复杂度为 $O(n^2)$ ）和高效排序（时间复杂度为 $O(n \log n)$ ）。本题给定 N 个整数，要求输出从小到大排序后的结果。请用不同的排序算法测试，注意有些算法无法拿到满分，由此同学们可以猜测一下 10 个测试用例的数据特征。

2.3.2 基本要求

请同学们自己随机生成不同规模的数据（例如 10, 100, 1K, 10K, 100K, 1M, 10K 正序, 10K 逆序），用不同的排序算法（快速排序, 归并排序, 堆排序, 选择排序, 冒泡排序, 直接插入排序, 希尔排序）分别对这些数据进行测试, 输出运行时间, 将结果写在实验报告中, 并总结各种排序算法的特点、时间复杂度、空间复杂度, 以及是否是稳定排序和部分排序。

2.3.3 数据结构设计

略

2.3.4 功能说明（函数、类）

```
// 快速排序
void quickSort(vector<int>& arr, int left, int right) {
    if (left >= right) return;
    int pivot = arr[(left + right) / 2];
    int i = left, j = right;
    while (i <= j) {
        while (arr[i] < pivot) i++;
        while (arr[j] > pivot) j--;
        if (i <= j) {
            swap(arr[i], arr[j]);
            i++;
            j--;
        }
    }
    quickSort(arr, left, j);
    quickSort(arr, i, right);
}

// 归并排序
void merge(vector<int>& arr, int left, int mid, int right) {
    vector<int> temp(right - left + 1);
    int i = left, j = mid + 1, k = 0;
    while (i <= mid && j <= right) {
        if (arr[i] <= arr[j]) temp[k++] = arr[i++];
        else temp[k++] = arr[j++];
    }
}
```

```

    }

    while (i <= mid) temp[k++] = arr[i++];
    while (j <= right) temp[k++] = arr[j++];
    for (int p = 0; p < k; p++) arr[left + p] = temp[p];
}

void mergeSort(vector<int>& arr, int left, int right) {
    if (left >= right) return;
    int mid = (left + right) / 2;
    mergeSort(arr, left, mid);
    mergeSort(arr, mid + 1, right);
    merge(arr, left, mid, right);
}

// 堆排序
void heapify(vector<int>& arr, int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    if (left < n && arr[left] > arr[largest]) largest = left;
    if (right < n && arr[right] > arr[largest]) largest = right;
    if (largest != i) {
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}

void heapSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = n / 2 - 1; i >= 0; i--) heapify(arr, n, i);
    for (int i = n - 1; i > 0; i--) {
        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }
}

```

```

}

// 选择排序
void selectionSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) minIndex = j;
        }
        swap(arr[i], arr[minIndex]);
    }
}

// 冒泡排序
void bubbleSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - 1 - i; j++) {
            if (arr[j] > arr[j + 1]) swap(arr[j], arr[j + 1]);
        }
    }
}

// 直接插入排序
void insertionSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
    }
}

```

```

        arr[j + 1] = key;
    }
}

// 希尔排序
void shellSort(vector<int>& arr) {
    int n = arr.size();
    for (int gap = n / 2; gap > 0; gap /= 2) {
        for (int i = gap; i < n; i++) {
            int temp = arr[i];
            int j;
            for (j = i; j >= gap && arr[j - gap] > temp; j -= gap) {
                arr[j] = arr[j - gap];
            }
            arr[j] = temp;
        }
    }
}

```

2.3.5 调试分析（遇到的问题 and 解决方法）

结果如下：（部分数据内容太少或太多，时间难以测量）（均使用乱序数据测量）

快速排序：

input_10:

运行时间(μs): 0***

input_100:

运行时间(μs): 0***

input_1000:

运行时间(μs): 0***

input_10000:

运行时间(μs): 0***

input_100000:

运行时间(μs): 16997***

input_1000000:

运行时间(μs): 141511***

归并排序：

input_10:

运行时间(μs): 0***

input_100:

运行时间(μ s): 0***
input_1000:
运行时间(μ s): 0***
input_10000:
运行时间(μ s): 0***
input_100000:
运行时间(μ s): 62056***
input_1000000:
运行时间(μ s): 500748***

堆排序:
input_10:
运行时间(μ s): 0***
input_100:
运行时间(μ s): 0***
input_1000:
运行时间(μ s): 0***
input_10000:
运行时间(μ s): 3091***
input_100000:
运行时间(μ s): 22270***
input_1000000:
运行时间(μ s): 409611***

选择排序:
input_10:
运行时间(μ s): 0***
input_100:
运行时间(μ s): 0***
input_1000:
运行时间(μ s): 0***
input_10000:
运行时间(μ s): 136781***
input_100000:
运行时间(μ s): 12397255***
input_1000000:
运行时间(μ s): No_Response***

冒泡排序:
input_10:
运行时间(μ s): 0***
input_100:
运行时间(μ s): 0***
input_1000:

运行时间(μ s): 0***
input_10000:
运行时间(μ s): 377375***
input_100000:
运行时间(μ s): 41638396***
input_1000000:
运行时间(μ s): No_Response***

插入排序:
input_10:
运行时间(μ s): 0***
input_100:
运行时间(μ s): 0***
input_1000:
运行时间(μ s): 0***
input_10000:
运行时间(μ s): 96326***
input_100000:
运行时间(μ s): 9248931***
input_1000000:
运行时间(μ s): No_Response***

希尔排序:
input_10:
运行时间(μ s): 0***
input_100:
运行时间(μ s): 0***
input_1000:
运行时间(μ s): 0***
input_10000:
运行时间(μ s): 9015***
input_100000:
运行时间(μ s): 31906***
input_1000000:
运行时间(μ s): 412691***

2.3.6 总结和体会

善用高效排序

2.4 序列

2.4.1 问题描述

给定 m 个数字序列，每个序列包含 n 个非负整数。我们从每一个序列中选取一个数字组成一个新的序列，显然一共可以构造出 n^m 个新序列。接下来我们对每一个新的序列中的数字进行求和，一共会得到 n^m 个和，请找出最小的 n 个和

2.4.2 基本要求

输入格式:

输入的第一行是一个整数 T ，表示测试用例的数量，接下来是 T 个测试用例的输入每个测试用例输入的第一行是两个正整数 m ($0 < m \leq 100$) 和 n ($0 < n \leq 2000$)，然后有 m 行，每行有 n 个数，数字之间用空格分开，表示这 m 个序列中的数字不会大于 10000

输出格式:

对每组测试用例，输出一行用空格隔开的数，表示最小的 n 个和

2.4.3 数据结构设计

```
// 前 i 行的最优解 a[N]
int m,n,a[N],b[N],c[N];
```

2.4.4 功能说明（函数、类）

```
void mer()//合并新行到之前结果并保留最优解 n 个
{
    priority_queue<pp, vector<pp>, greater<pp> > heap;
    for(int i = 0; i < n; i++)heap.push({a[0] + b[i], 0});
    for(int i = 0; i < n; i++)
    {
        pp t=heap.top();
        heap.pop();
        int s=t.first,p=t.second;
        c[i]=s;
        heap.push({s - a[p] + a[p+1], p+1});
    }
    for(int i = 0; i < n; i++) a[i]=c[i];
}
```

2.4.5 调试分析（遇到的问题 and 解决方法）

问题：MLE

解决方案：逐行求解

2.4.6 总结和体会

不能一次求解，可以逐步求解

2.5 三数之和

2.5.1 问题描述

给你一个整数数组 `nums`，判断是否存在三元组 `[nums[i], nums[j], nums[k]]` 满足 $i \neq j$ 、 $i \neq k$ 且 $j \neq k$ ，同时还满足 $nums[i] + nums[j] + nums[k] == 0$ 。请你返回所有和为 0 且不重复的三元组，每个三元组占一行。

2.5.2 基本要求

输入描述

2 行：第 1 行一个整数，表示数组元素个数；第 2 行输入一组整数，中间以空格分隔。

输出描述

输出所有和为 0 的三元组，每个三元组一行，中间以空格分隔。对于每一个三元组，你需要按从小到大的顺序依次返回三个元素；对于所有三元组，你需要按三元组中最小元素从小到大的顺序依次打印每一组三元组。

2.5.3 数据结构设计

```
char vis[10000010]; // 桶排序
int num[3010]; // 登记不重复的数字
```

2.5.4 功能说明（函数、类）

```
while(num[l]<0){
    int r=cnt-1;
    for(;num[r]>0;r--){
        // cout<<"consider: "<<num[l]<<" & "<<num[r]<<endl;
        int diff=-(num[l]+num[r]);
        if(diff>num[r]||diff<num[l])
            continue;

        vis[OFFSET+num[l]]--;
        vis[OFFSET+num[r]]--;

        if(vis[OFFSET+diff])
            printf("%d %d %d\n",num[l],-(num[l]+num[r]),num[r]);

        // 这里从小到大
        vis[OFFSET+num[l]]++;
        vis[OFFSET+num[r]]++;
    }
}
```

```
    }  
    L++;  
}  
// 遍历最大、最小值，利用桶排序查看是否存在使得三数和为 0 的值
```

2.5.5 调试分析（遇到的问题和解决方法）

问题：RE

解决方案：数组不够大，开大一点就好了

问题：重复结果

解决方案：只考虑最大\最小值，从而避开对 0 的考虑和重复

2.5.6 总结和体会

逐个排查错误，逐步完成部分功能

3. 实验总结

本次实验涵盖了多个与排序和序列处理相关的算法问题，包括逆序对数的计算、最大数的排列、不同排序算法的性能比较、最小和的求解以及三数之和的问题。通过这些问题，我们深入理解了排序算法的原理、应用场景以及优化方法。以下是本次实验的主要总结和体会：

1. 逆序对数问题

核心思想：通过归并排序的过程计算逆序对数。归并排序在合并两个有序子序列时，可以高效地统计逆序对的数量。在合并过程中，如果右子序列的元素小于左子序列的当前元素，则左子序列剩余的所有元素都与该右子序列元素构成逆序对。需要确保合并后的有序序列覆盖原序列，否则递归回到上层时会出错。逆序对数的计算与排序过程密切相关，归并排序是解决此类问题的经典方法。

2. 最大数问题

核心思想：通过贪心算法对数字进行重新排列，使得拼接后的数字最大。

关键点：

定义比较函数 $\text{comp}(s1, s2)$ ，判断 $s1 + s2$ 和 $s2 + s1$ 的大小关系。需要证明比较函数的传递性，以确保排序结果的正确性。贪心算法的正确性依赖于比较函数的定义，必须确保其满足传递性和自反性。

3. 排序算法性能比较

核心思想：通过实验比较不同排序算法（如快速排序、归并排序、堆排序、选择排序、冒泡排序、直接插入排序、希尔排序）在不同数据规模和数据特征下的性能。

时间复杂度：高效排序算法（如快速排序、归并排序、堆排序）的时间复杂度为 $O(n \log n)$ ，而简单排序算法（如选择排序、冒泡排序、直接插入排序）的时间复杂度为 $O(n^2)$ ，希尔排序的复杂度大概是 $O(n^{1.3-1.5})$ 。

空间复杂度：归并排序需要 $O(n)$ ，快速排序 $O(\log n)$ - $O(n)$ ，其他为 $O(1)$ 原地工作。

稳定性：归并排序和直接插入排序、冒泡排序是稳定排序，其他排序为不稳定排序。

4. 最小和问题

核心思想：通过逐行求解的方法，利用优先队列（堆）维护当前的最小和。每行数据与之前的结果合并，保留前 n 个最小和。使用优先队列优化合并过程，避免一次性求解导致内存

溢出（MLE）。对于大规模数据，逐步求解和优化空间复杂度是关键。

5. 三数之和问题

核心思想: 通过遍历数组, 利用双指针和桶排序的方法, 高效地找到所有满足条件的三元组。遍历时固定最小值和最大值, 利用桶排序检查是否存在满足条件的中间值。需要处理重复结果的问题, 确保每个三元组只被记录一次。双指针和桶排序的结合可以有效减少时间复杂度, 同时需要注意边界条件和重复结果的过滤。

总体体会

不同问题需要选择不同的算法, 理解算法的原理和适用场景是解决问题的关键。在解决问题时, 不仅要考虑算法的正确性, 还要关注时间复杂度和空间复杂度的优化。在实现算法时, 可能会遇到各种问题 (如内存溢出、重复结果等), 需要通过调试和分析逐步解决。通过实验验证算法的性能, 并总结各种算法的优缺点, 有助于加深对算法的理解。